

Context Awareness Engine: How It Works & Calculates

Complete mathematical and algorithmic specification for cross-plane evidence correlation, directed graph synthesis, anti-double-counting, and deterministic provenance risk scoring.

1 Executive Summary: The Context Problem

Standard security tools evaluate checks in total isolation. A signature check notes an unsigned commit; a file check notes a .gitignore rule; a similarity checker flags a matched code block. In a high-stakes hackathon audit, this creates two catastrophic failure modes:

1. Fragmented Narrative Blindness

Judges receive 20+ isolated rows with no way to tell whether an unsigned commit and an uncommitted source file are part of a coordinated code smuggling attack or two harmless unrelated oversights.

2. Double-Counting Injustice

If a single stolen file triggers a similarity flag, a concealment flag, and a pre-T0 timestamp warning, naive scanners penalize the team 3 separate times for the exact same event.

The **Context Awareness Engine** sits above all four analyzer planes (GPG claim plane, .gitignore system plane, timestamp claim plane, and GitHub server plane). It translates raw findings into a directed evidence graph, correlates related entities, eliminates double-counting, and computes a deterministic risk verdict.

2 4-Stage Architectural Pipeline

Pipeline Execution Flow:

Raw Findings (23 Checks) + Gitignore Blame View + Similarity Hits

- **Stage 1 (Normalizer)**: Ingests & maps findings into uniform EvidenceItem entities.
- **Stage 2 (EvidenceGraph)**: Constructs in-memory directed graph with typed nodes & edges.
- **Stage 3 (Correlator)**: Applies Rules 1–7, performs single-event grouping, builds narrative trees.
- **Stage 4 (Risk Engine)**: Calculates deterministic status badge, risk level, and confidence.

Module	Source File	Primary Responsibility & Algorithmic Function
Normalization	core/context/normalizer.py	Extracts entities (file path, commit SHA, author, email, timestamp, key ID). Enforces clone-safety by honoring evaluable=False so machine-local configs are never falsified.
Evidence Graph	core/context/graph.py	In-memory graph storing 9 typed nodes (File, Commit, Author, etc.) and 10 typed directed edges (AUTHORED_BY, INTRODUCED_BY, MATCHES, etc.).
Correlation Engine	core/context/correlator.py	Executes Rules 1 to 7. Groups related items by file and commit. Distinguishes benign build artifacts from real source concealment. Consolidates redundant signals into single chains.
Risk Engine	core/context/risk.py	Evaluates chain severity rankings. Emits overall status, confidence level, key drivers, and unresolved advisory signals using strict deterministic logic (zero LLM).

3 The 7 Deterministic Correlation Rules

Every connection made by the engine is governed by seven deterministic rules. If a rule condition is met, the engine creates or merges a `CorrelationChain`:

Rule	Trigger Condition	Graph Traversal & Linkage	Severity & Badge	Narrative Output
Rule 1	Similarity match $\geq 75\%$ AND source repo date < \$T_0\$.	(File) –[MATCHES]→ (ExtRepo) –[PREDATES]→ (T0)	hard_flag CONCERN	Pre-existing code importation / plagiarism detected.
Rule 2	File similarity match > threshold AND Git blame attribution available.	(File) –[INTRODUCED_BY]→ (Commit) –[AUTHORED_BY]→ (Author)	flag WARNING	Attributed code match with exact author, commit SHA, and date.
Rule 3	Author != GPG signer OR signer not in team roster.	(Commit) –[AUTHORED_BY]→ (Author) & (Commit) –[SIGNED_BY]→ (Key)	flag / hard_flag CONCERN	Identity spoofing, shared account, or unregistered key usage.
Rule 4	File hidden in .gitignore or index flag (skip-worktree).	(File) –[HIDDEN_BY]→ (Rule) –[INTRODUCED_BY]→ (Commit)	info (Build artifact) flag (Source code)	Separates benign vendor folders (dist/, node_modules/) from source concealment.
Rule 5	Commit timestamp < \$T_0\$ OR Author/Committer delta > 24h.	(Commit) –[OCCURRED_AT]→ (Timestamp) –[OUT_OF_BOUNDS]	hard_flag / flag WARNING	Out-of-bounds commit, pre-baked history, or timestamp manipulation.
Rule 6	Commit in local clone missing from GitHub OR force-push detected.	(Commit) –[PUSHED_IN]→ (GitHub Events)	flag / hard_flag WARNING	Unpushed work or history rewriting to conceal prior development.
Rule 7	Multiple signals point to the same root file or commit entity.	Consolidates all items into 1 CorrelationChain .	Worst Severity of component items	Anti-Double-Counting : single event with supporting evidence.

4 Anti-Double-Counting & Single-Event Grouping

Anti-double-counting is achieved through entity-keyed grouping and identity consumption:

Mathematical Formulation

Let $E = \{e_1, e_2, \dots, e_n\}$ be the set of raw evidence items.

The correlator maps E into an equivalence relation over root entities R :

$$\text{Chain}(r) = \{e \in E \mid \text{entity}(e) = r\}$$

$\text{Deduplicated Count} = |E| - |\text{Chains}|$

The total penalty applied to a

Benign Artifact Filter Formula

A concealed path P is classified as **Benign** if:

$$\text{dir}(P) \cap \{\text{node_modules}, \text{dist}, \text{build}, \text{.env}\} \neq \emptyset \text{ OR } \text{ext}(P) \in \{\text{.min.js}, \text{.lock}, \text{.map}, \text{.env}\}$$

If Benign $\rightarrow$ marked **CLEAN**

(Severity = info).

team reflects $\$ \backslash \text{Chains} \mid \$$, ensuring an incident is penalized exactly once.

If Real Source File ($\$P \backslash \text{in} \backslash \text{Source} \mid \$$) $\$ \backslash \text{to} \$$ marked **ACTIVE CONCEALMENT**.

5 Deterministic Risk Scoring Formula

The overall verdict and confidence level are calculated deterministically:

Overall Status Calculation:

- If $\$ \backslash \text{exists } c \backslash \text{in} \backslash \text{Chains} \mid \$$ with $\$ \backslash \text{severity} \mid (c) = \backslash \text{ttt} \{ \text{hard} \backslash _ \text{flag} \} \$$ → 🚨 **HIGH-RISK**
PROVENANCE CONCERN (Risk: HIGH, Conf: HIGH)
- Else if $\$ \backslash \text{exists } c \backslash \text{in} \backslash \text{Chains} \mid \$$ with $\$ \backslash \text{severity} \mid (c) = \backslash \text{ttt} \{ \text{flag} \} \$$ → ⚠️ **EVIDENCE**
CORRELATION: MEDIUM RISK (Risk: MEDIUM, Conf: HIGH if >1 flag else MED)
- Else → ✅ **VERIFIED / CLEAN** (Risk: LOW, Conf: HIGH)

6 Terminal Output & Visual Narrative Interpretation

Case 1: Clean Hackathon Submission

When all checks pass and all commits, timestamps, and files align with hackathon rules:

```
=====
                        CONTEXT AWARENESS SUMMARY
=====
Status: ✔ VERIFIED / CLEAN [Confidence: HIGH]
Correlation Overview:
• No anomalous chains detected across all analysis modules.
• All commits, keys, files, and timestamps correlate consistently with hackathon rules.
=====
```

Case 2: Multi-Signal Provenance Tampering

When a participant smuggled pre-existing code and attempted concealment:

```
=====
                        CONTEXT AWARENESS SUMMARY
=====
Status: 🚨 HIGH-RISK PROVENANCE CONCERN [Confidence: HIGH]
Correlation Overview:
• 2 anomalous chain(s) identified from 7 raw evidence item(s)
• Consolidated 5 redundant/multi-plane signal(s) into single event chains

[CHAIN_001] 🚨 Code Provenance Match: src/auth.py [CONCERN]
[FILE] src/auth.py
├─ Similarity Match: 94% identical to acme/oauth-starter/auth.ts
├─ External Source Date: 2024-01-10 (Predates hackathon window)
├─ Introduced in Commit: a1b2c3d (committed 2h after T0)
└─ Concealment Signal: Concealed file src/auth.py hidden by 'src/auth.py' committed by Alice (e4f5a6b)

[CHAIN_002] ⚠ Commit Integrity Analysis: 9f8e7d6 [WARNING]
[COMMIT] 9f8e7d6 (Bob Hacker)
├─ [IDENTITY] Signer Charlie does not match commit author Bob
└─ [SIGNATURE] Commit 9f8e7d6 signature valid=False by Charlie

Assessment & Key Drivers:
• Multi-signal correlation on src/auth.py (4 linked signals, rule: Rule 1: Similarity + Pre-hackathon source)
• Multi-signal correlation on Commit 9f8e7d6 (2 linked signals, rule: Rule 3: Identity & Signer correlation)
=====
```

7 JSON Schema Structure

When running with `--format json`, HACKPROOF emits the full evidence graph and correlation chains:

```
{
  "repo_path": "priyanshu22102006/HACKPROOF",
  "summary": { "passed": 22, "failed": 1, "hard_flag": 0, "flag": 0, "info": 23 },
  "context_awareness": {
    "overall_status": "VERIFIED / CLEAN",
    "risk_level": "LOW",
    "confidence": "HIGH",
    "total_raw_evidence_count": 23,
    "total_chains_count": 18,
    "chains": [
      {
        "chain_id": "CHAIN_001",
        "title": "Standard Build / Env Artifact Ignored: dist/bundle.min.js",
        "severity": "info",
```

```

    "root_entity": "dist/bundle.min.js",
    "status_badge": "CLEAN",
    "rule": "Rule 4: Hidden tracking + File analysis",
    "narrative": [
      "[FILE] dist/bundle.min.js",
      "└─ Classification: BENIGN BUILD ARTIFACT",
      "└─ Rule Origin Commit: 471668c by Pritim Mondal"
    ]
  },
  "graph": {
    "nodes": [
      { "id": "file:dist/bundle.min.js", "type": "File", "attrs": { "path": "dist/bundle.min.js" } },
      { "id": "commit:471668c", "type": "Commit", "attrs": { "sha": "471668c" } },
      { "id": "author:mondalpritim14@gmail.com", "type": "Author", "attrs": { "name": "Pritim Mondal" } }
    ],
    "edges": [
      { "source": "commit:471668c", "target": "file:dist/bundle.min.js", "type": "INTRODUCED_BY" },
      { "source": "commit:471668c", "target": "author:mondalpritim14@gmail.com", "type": "AUTHORED_BY" }
    ]
  }
}

```